

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ingeniería Mecánica y Metalúrgica

Manual de Uso de MuJoCo

Introducción al uso y funcionamiento del simulador

Vittorio Cherubini Grandón
Profesor: Sergio Aguilera Marinovic

Fecha de elaboración: 2026

Índice

1. Introducción a MuJoCo **1**
- 1.1. Qué es MuJoCo? . . . 1
- 1.2. Por que MuJoCo? . . . 1
- 1.3. Estructura del manual . . . 1
- 1.4. Documentación . . . 1
- 1.5. IA . . . 2

2. Instalación **3**
- 2.1. Sistema Operativo . . . 3
- 2.2. Instalar MuJoCo para Python . . . 3
- 2.3. MuJoCo Menagerie (obtención de modelos 3D) . . . 4

3. Estructura del directorio y repositorio **5**
- 3.1. Directorios . . . 5
- 3.2. Repositorio . . . 5

4. Ejemplo 1: Simulador **6**
- 4.1. Parte 1: Abrir ventana y escena vacía . . . 6
- 4.2. Parte 2: Crear código base de Python . . . 6
- 4.3. Parte 3: Añadir objetos a la escena . . . 7

5. Ejemplo 2: Añadir un Robot **11**
- 5.1. Parte 1: Conseguir el modelo del brazo . . . 11
- 5.2. Parte 2: Cómo se compone un robot en MuJoCo? . . . 12
  - 5.2.1. Cómo se componen los robots en Mujoco (Explicación Nivel 1) . . . 12
  - 5.2.2. Carpeta de assets . . . 13
  - 5.2.3. scene.xml . . . 13
  - 5.2.4. fr3.xml . . . 13
  - 5.2.5. fr3.xml - Valores por defecto . . . 13
  - 5.2.6. fr3.xml - Assets . . . 14
  - 5.2.7. fr3.xml - Definir el robot y articulaciones (Parte 1) . . . 14
  - 5.2.8. Cómo se componen los robots en Mujoco (Explicación Nivel 2) . . . 14
  - 5.2.9. fr3.xml - Definir el robot y articulaciones (Parte 2) . . . 15
  - 5.2.10. fr3.xml - Actuadores . . . 16
  - 5.2.11. fr3.xml - Exclusión de contacto . . . 16
- 5.3. Parte 3: Incluir el brazo en nuestra escena . . . 16
  - 5.3.1. e2.xml . . . 16
  - 5.3.2. e2.py . . . 17
  - 5.3.3. Controlando el robot (UI) . . . 17
  - 5.3.4. Controlando el robot (código) . . . 18

6. Ejemplo 3: Ensamble de Robots **19**
- 6.1. Descargar el modelo de la mano. . . . 19
- 6.2. Escena . . . 19
- 6.3. e3.py . . . 19

7. Conclusión **22**

8. Fuentes y Referencias **23**

1. Introducción a MuJoCo



Figura 1: Logo de MuJoCo (DeepMind, 2022)

1.1. Qué es MuJoCo?

Oficialmente es una librería desarrollada para C/C++, pero también se puede usar como librería de Python. Es un simulador orientado a sistemas articulados que deben interactuar por contacto con su entorno (MuJoCo es un acrónimo de *Multi-Joint dynamics with Contact*), inicialmente desarrollado por Roboti LLC y posteriormente adquirido por DeepMind (Google), quienes lo hicieron *open source* el año 2022 (DeepMind, 2022).

1.2. Por que MuJoCo?

En corto, MuJoCo está entre los mejores simuladores disponibles para escenarios que requieren exactitud física. Se usa mucho para simulaciones de fuerzas de contacto y colisiones, especialmente para sistemas articulados. Además, es muy eficiente computacionalmente.

1.3. Estructura del manual

Luego de la instalación, a lo largo del manual se irá construyendo conocimiento guiándose por diferentes ejemplos o tutoriales. Cada ejemplo se puede seguir paso a paso y tiene sus propios objetivos pedagógicos. Esto es para que el lector vaya familiarizándose progresivamente con el simulador.

Para cada ejemplo, se adjunta explícitamente el código de cada sección. La idea es que asi se motiven para leer el código y entenderlo, aprovechando que está decentemente comentado.

Se dará por sentado que el usuario está familiarizado con conceptos teóricos de control, dinámica y programación. Por ello, no se profundizará en estos temas en este manual.

Los códigos y ejemplos de este manual están disponibles en el siguiente repositorio:
https://github.com/vittocher/mujoco-manual

1.4. Documentación

El recurso más utilizado para la elaboración de este manual es la documentación oficial de Mujoco. Por ese motivo, es recomendado consultarla en caso de dudas, aclaraciones o para buscar

más información respecto al funcionamiento del simulador. El enlace directo se encuentra en las referencias del presente manual.

1.5. IA

Se puede usar IA con Mujoco porque está bien documentado. Sin embargo, hay varias formas de lograr los mismos objetivos y las recomendaciones que te otorgan las herramientas de Inteligencia Artificial no siempre son las mejores. En general, simulación es un nicho bien documentado pero sigue siendo pequeño, por lo que la IA no funciona tan bien como en otros nichos más populares (como desarrollo web, por ejemplo).

2. Instalación

2.1. Sistema Operativo

Para esta parte del manual, se asumirá que el usuario trabaja con Linux Ubuntu. Para la elaboración del presente documento se utilizó **Linux Ubuntu 22.04**. MuJoCo **igualmente se puede utilizar en Windows y MacOS**. Sin embargo, los pasos de instalación pueden variar..

2.2. Instalar MuJoCo para Python

Esta instalación se hace directamente con *pip*. En caso de no tener pip, se debe instalar. Este último proceso es simple y ampliamente documentado en línea, así que no se explicará en este documento.

A continuación, los pasos para instalar MuJoCo como librería de Python (se puede hacer desde un venv si quieres):

1. Abrir el terminal
2. Instalar con pip → *pip install mujoco*

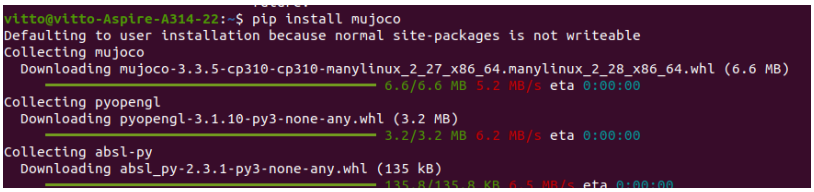


Figura 2: Output esperado del paso 2

3. Revisar versión de MuJoCo instalada → *pip show mujoco*



Figura 3: Output esperado del paso 3

4. Abrir una ventana simple de MuJoCo → *python3 -m mujoco.viewer*
   Si la instalación salió bien, luego de correr el comando en la terminal se debería abrir una ventana simple de MuJoCo. Con eso, está lista la instalación.



Figura 4: *Mujoco viewer* vacío o simple

2.3. MuJoCo Menagerie (obtención de modelos 3D)

MuJoCo Menagerie es un repositorio de GitHub, publicado por el mismo DeepMind, que contiene muchos modelos de alta fidelidad de distintos sistemas articulados de la industria. Además, vienen listos para ser usados en simulación. Esto lo hace un recurso muy útil y que se seguirá usando durante el resto del manual.

El link directo al repositorio es el siguiente: https://github.com/google-deepmind/mujoco_menagerie.

Para instalarlo, basta con clonar el repositorio en una carpeta aparte.

Para utilizar los modelos contenidos, basta con mover su carpeta correspondiente del repositorio al directorio donde se esté trabajando. Cada carpeta viene con una imagen de previsualización y el Menagerie está bien organizado, así que el proceso de selección es simple.

Menagerie significa "colección de fieras" en español.

3. Estructura del directorio y repositorio

3.1. Directorios

Para hacer un proyecto en MuJoCo, es recomendable crear un directorio que almacene todos los códigos o modelos necesarios.

Esta sección está sirve para aclarar cómo estructurar un proyecto desde 0. En estricto rigor, MuJoCo es una librería por lo que se puede usar dónde y cómo sea (a diferencia de trabajar con ROS2, por ejemplo, que te obliga a usar paquetes). Sin embargo, mantener una estructura cohesiva ayuda al programador a navegar mejor su código y facilita el trabajo colaborativo.

Dicho esto, con MuJoCo se trabaja principalmente con dos tipos de archivos: **scripts** (código en Python, C o C++) y **modelos o escenas** (archivos .xml que describen robots o entornos completos). Si modelas tus propios robots, también trabajarás con **.obj** y **.stl**, pero no se tocará ese tema en este manual.

Los modelos de MuJoCo Menagerie vienen en una carpeta con todos los assets necesarios, un .xml del robot y un .xml de una escena simple para visualizar el robot. Esto es útil para visualizar el robot y que interactúe con un entorno simple. Si se quiere trabajar con más de un robot en una escena, se deben cambiar estos archivos. Con esto en mente, un repositorio normal se suele ver (aproximadamente) así:

```
directorio del proyecto/
    ├── directorio de assets/
    │       └── modelo/
    │               ├── modelo_assets.xml/
    │               ├── modelo_actuators.xml/
    │               ├── modelo_base.xml/
    │               ├── parte1.obj/
    │               ├── parte2.obj/
    │               ├── parte1.stl/
    │               └── parte2.stl/
    ├── scripts.py/
    └── escenas.xml/
```

'Scripts', *'escenas'* y *'directorios de modelos'* se encuentran en plural ya que puede haber más de uno. Cada .xml dentro del directorio de modelo se puede construir dentro del directorio del modelo en Mujoco Menagerie. Más sobre esto en una sección futura.

Esta estructura no es obligatoria ni se usa siempre, pero es un buen acercamiento inicial a cómo suelen ser los directorios de proyectos de mujoco. Además, es la que se usará por el resto del manual.

Para seguir con el resto del manual, es recomendable crear un directorio vacío y llamarlo como se estime conveniente, como por ejemplo `mujoco_manual_uc/`.

3.2. Repositorio

Todos los códigos del manual se encuentran en el siguiente repositorio:

https://github.com/vittocher/mujoco-manual

Cada **ejemplo** estará contenido dentro de su propio **directorio** dentro del repositorio.

4. Ejemplo 1: Simulador

El objetivo de este ejemplo es aprender como desplegar un entorno de simulación, interactuar con la ventana del simulador y crear una escena con objetos estáticos y dinámicos.

4.1. Parte 1: Abrir ventana y escena vacía

Primero, vamos a crear una escena en .xml y un código en Python que nos abra una ventana simple del simulador.

Dentro de nuestro directorio de trabajo, vamos a crear un archivo llamado `"e1_pt1.xml"` que contendrá una escena completamente vacía. Dentro de él, escribimos:

```
<mujoco model="empty_scene">
</mujoco>
```

Con esto, ya definimos nuestra primera escena de MuJoCo! Realmente no es una escena, sino un modelo. Sin embargo, cuando hablemos de escena, nos estaremos refiriendo al modelo principal que procesa el código y que reúne a todos los modelos de la simulación. Por ahora sólo tiene nombre, pero más adelante vamos a introducir objetos a esta escena.

4.2. Parte 2: Crear código base de Python

Dentro de nuestro directorio de trabajo, crearemos un archivo de Python llamado `"e1_pt2.py"` que usará nuestra escena vacía.

Dentro de él, escribimos:

```
# Importar librería de MuJoCo
import mujoco
import mujoco.viewer
from pathlib import Path

# Cargar escena que creamos en el paso anterior
xml_path = Path(__file__).resolve().parent / "e1_pt1.xml"
model = mujoco.MjModel.from_xml_path(str(xml_path))
data = mujoco.MjData(model)

# Abre el viewer
with mujoco.viewer.launch_passive(model, data) as viewer:
    # Se inicia el loop inicial de simulación
    while viewer.is_running():
        #############
        ## Lógica ##
        #############
        # Actualizar la información del modelo o escena
        mujoco.mj_step(model,data)
        # Sincronizar la ventana con la simulación
        viewer.sync()
```

Dentro del *while loop* se ejecuta la lógica principal del código del simulador. Si una parte de tu código se ejecuta en tiempo real, debería ir incluida en este while loop. Si se ejecuta sólo una vez,

debería ir antes del *loop* para evitar redundancias; esto es útil para cualquier cálculo inicial necesario que no quieras repetir en cada iteración.

Al final del *loop*, se actualiza la información del modelo con `mujoco.mj_step()` en base a la lógica del código y los cambios que hayan ocurrido en el tiempo y luego se actualiza el visualizador con `viewer.sync()`.

Si ejecutamos el código anterior, se nos debería abrir una ventana como la siguiente:



Figura 5: Ventana esperada del ejemplo 1.1

En esta ventana no podemos hacer nada todavía. Para salir, basta con presionar el botón: "***Quit***"

4.3. Parte 3: Añadir objetos a la escena

Ahora vamos a añadir objetos a la escena. Para ello, crearemos un nuevo archivo llamado `e1_pt3.xml`. Parte igual que la escena anterior, sólo que ahora le pondremos objetos dentro y un timestep para que la simulación se mueva de forma visualmente coherente.

Dentro del archivo, escribimos:

```
<mujoco model="Simple Scene">

  <!-- Ajustes del compilador -->
  <compiler angle="radian" meshdir="assets" coordinate="local" autolimits="true"/>
  <option integrator="implicitfast"/>

  <!--Textura del mundo (skybox y piso)-->
  <asset>
    <texture type="skybox" builtin="gradient" rgb1="0.3 0.5 0.7" rgb2="0 0 0" width="512" height="30
    <texture type="2d" name="groundplane" builtin="checker" mark="edge" rgb1="0.2 0.3 0.4" rgb2="0.1
```

```
    markrgb="0.8 0.8 0.8" width="300" height="300"/>
  <material name="groundplane" texture="groundplane" texuniform="true" texrepeat="5 5" reflectance
</asset>

<!-- Objetos en escena -->
<worldbody>
  <!-- Luz global -->
  <light pos="0 0 1.5" dir="0 0 -1" directional="true"/>
  <!-- Piso -->
  <geom name="floor" size="0 0 0.05" type="plane" material="groundplane"/>

  <!-- Cubo estático -->
  <geom name="static_cube" type="box" pos="0.265 0 0.78" size="0.2 0.2 0.02"
        rgba="0.8 0.8 0.2 1" contype="1" conaffinity="1" condim="4"/>

  <!-- Cilindro dinámico -->
  <body name="dynamic_cylinder" pos="0 0 2">
      <freejoint/>   <!-- permite movimiento libre -->
      <geom type="cylinder" size="0.15 0.01" rgba="0.2 0.2 0.8 1"
            mass="1" contype="1" conaffinity="1" condim="4"/>
  </body>
</worldbody>

</mujoco>
```

Ahora vemos varios elementos en el XML.

La primera parte tiene ajustes de compilador. <***compiler***> indica cómo MuJoCo interpreta el XML al cargarlo (por ejemplo, angle=radianïndica que usamos radianes en vez de *degrees*). <***option***> tiene parámetros del motor de física (integrator = ïmplicitfast"setea el integrador que usa el simulador).

La segunda parte, contenida dentro de <***asset***> tiene los ajustes visuales del skybox y la textura del piso.

La tercera parte, contenida dentro de <***worldbody***> , tiene todos los objetos que existen físicamente en la escena, junto con su configuración. Puedes ver los distintos tipos de objetos y sus propiedades (como su pos inicial, nombre, tipo, etc). El cubo, al ser estático, se define como sólo una geometría capaz de colisiones. El cilindro, al ser dinámico, se define como un cuerpo con .ªrticulación libre.

Primero se define el cuerpo y, dentro de él, una **articulación** a la cual se le **asocia una geometría**. En este caso, la articulación es libre, por lo que la geometría definida después de ella se puede mover libremente. Luego veremos otro tipo de articulaciones o *joints* que definen distintos tipos de movimiento, en distintos ejes y con diferentes limitaciones.

Para manejar las colisiones, se usan los atributos *contype*, *conaffinity* y *condim*. Las primeras dos definen la "máscara"de colisión. Básicamente, si dos objetos tienen distintas ***conaffinity*** y ***contype***, no colisionan entre si. Por otro lado, ***condim*** define las dimensiones en las cuales ocurre la colisión y sólo acepta *1, 3, 4* y *6*. Si *condim=1*, las colisiones sólo aplican la fuerza normal. Si *condim=3* , se añade ***fricción*** a la colisión. Si *condim=4*, se añade **resistencia al giro** sobre la normal de contacto, lo que es útil para simulaciones de contacto de cuerpos redondos. Si *condim=6*, se añaden las dos anteriores al mismo tiempo, lo que es más caro computacionalmente. Por eso usamos *condim=4*.

Ahora, creamos el script llamado `e1_pt3.py` que abre la escena e implementa un *timestep*.

Dentro del archivo, escribimos:

```
# Importar librería de MuJoCo
import mujoco
import mujoco.viewer
from pathlib import Path
import time

# Cargar escena que creamos en el paso anterior
xml_path = Path(__file__).resolve().parent / "e1_pt3.xml"
model = mujoco.MjModel.from_xml_path(str(xml_path))
data = mujoco.MjData(model)
print("Timestep de la simulación:", model.opt.timestep)
# Abre el viewer
with mujoco.viewer.launch_passive(model, data) as viewer:
# Se inicia el loop inicial de simulación
    while viewer.is_running():
        #############
        ## Lógica ##
        #############

        # Actualizar la escena
        step_start = time.time()
        mujoco.mj_step(model,data)
        viewer.sync()

        # Esperar para que el paso dure lo mismo en tiempo real que en simulación
        time_until_next = 2*model.opt.timestep - (time.time() - step_start)
        if time_until_next > 0:
            time.sleep(time_until_next)
```

Al ejecutarlo, vemos la escena que acabamos de crear, con un piso y un objetos dinámicos y estáticos.

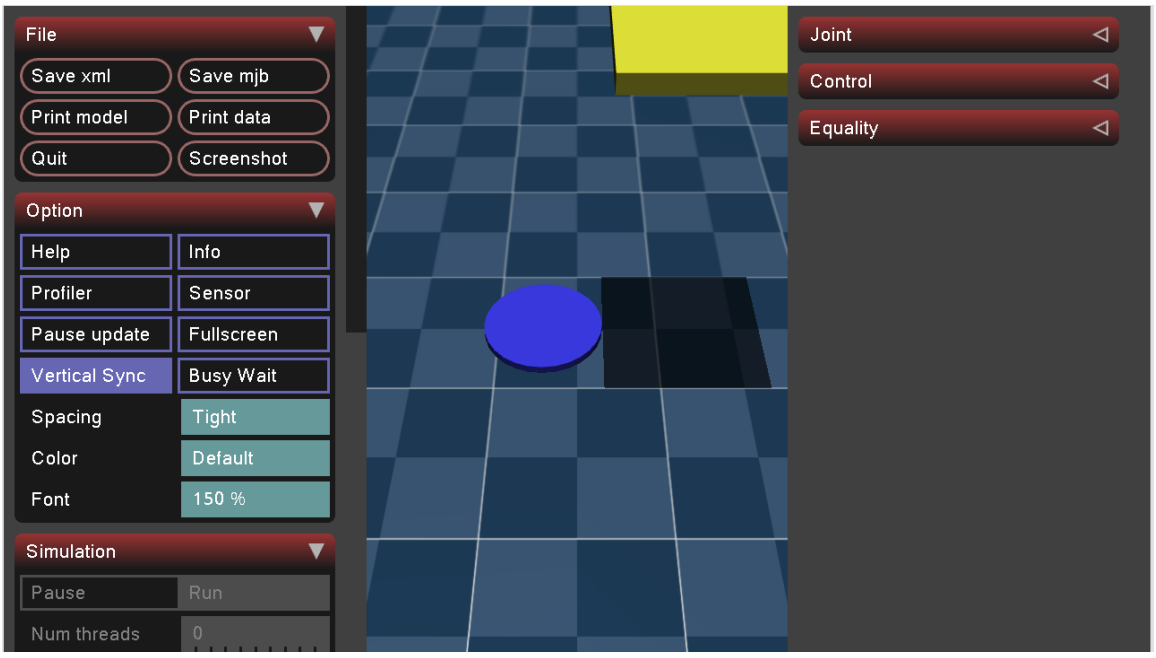


Figura 6: Ventana esperada del ejemplo 1.3

Ahora que sí tenemos una simulación real, podemos empezar a ver cómo interactuar con la ventana.

Con *click izquierdo* y *derecho* movemos la cámara. Con la *rueda del ratón* hacemos zoom.

En las opciones de la izquierda, si bajamos a la sección de "*Simulation*", podemos reiniciar la simulación desde 0 para volver a ver el cilindro caer. Si todo está bien, durante la caída deberíamos ver el cilindro azul colisionar con el cubo amarillo y caer al piso.

Con esto, terminamos el primer ejemplo! Ahora ya sabemos crear una escena con objetos estáticos y dinámicos, además de movernos en ella y reiniciar el tiempo de la simulación.

5. Ejemplo 2: Añadir un Robot

Ahora que ya conocemos un poco del simulador, añadamos un robot!

El objetivo de este ejemplo es aprender cómo añadir y controlar un robot en Mujoco.

5.1. Parte 1: Conseguir el modelo del brazo

Para este ejemplo, usaremos el brazo robótico FR3 de Franka Emika.

De dónde lo obtendremos? – Del Mujoco Menagerie!

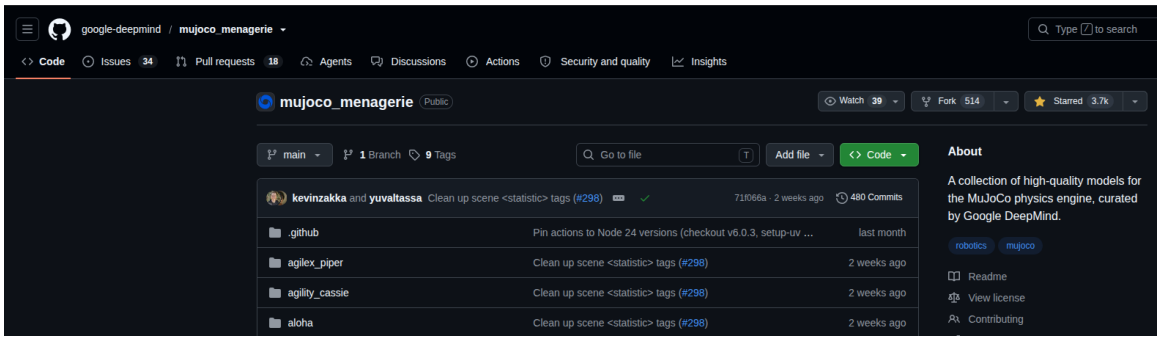


Figura 7: MuJoCo Menagerie

En este repositorio hay modelos de varios robots diferentes; carros, brazos y manos, entre varios otros tipos de robot. Cada modelo se basa en robots de la vida real, de distintas empresas y funcionalidades, cada uno disponible en el mercado. La gracia de esto es poder tener "gemelos digitales"de estos robots, para poder trabajar con ellos de forma digital antes o durante del trabajo en la vida real.

Volviendo al tutorial, debemos clonar o descargar el repositorio para poder acceder a sus archivos. En concreto, ahora mismo sólo nos interesa la carpeta llamada `"franka_fr3"`.



Figura 8: Como aparece el directorio en el repositorio

Ahora, debemos **copiar** esta carpeta de nuestra descarga del Menagerie y **pegarla** en nuestro directorio de trabajo.

5.2. Parte 2: Cómo se compone un robot en MuJoCo?

Ahora tenemos un robot digital en nuestras manos! Pero cómo se compone este robot en Mujoco? Para responder estas preguntas, veamos que contiene el directorio.

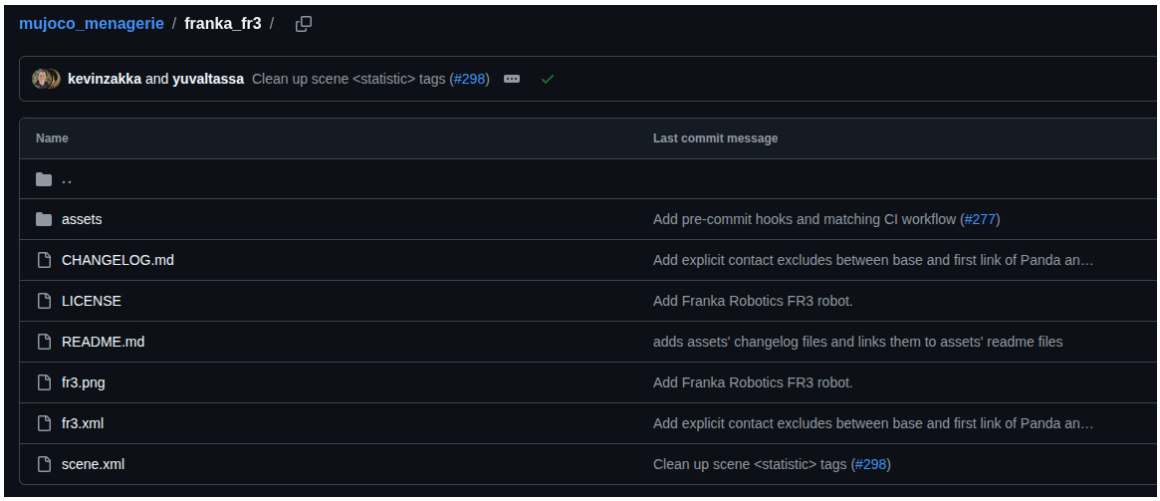


Figura 9: Directorio del robot que nos interesa

De aqui nos interesa la carpeta **assets** y los archivos **fr3.xml** y **scene.xml**.

5.2.1. Cómo se componen los robots en Mujoco (Explicación Nivel 1)

Dicho de forma simple, los robots se dividen en cuerpos y articulaciones.

Los **cuerpos** son las partes del robot. Las **articulaciones** unen estos cuerpos y definen cómo se puede mover un objeto en relación al otro.

Viendo el ejemplo de un dedo humano, podemos dividirlo en 3 partes o falanges. En Mujoco, cada falange es un cuerpo propio, con geometría propia en el mundo simulado. Estos serían sus **cuerpos**.

Siguiendo el ejemplo, para construir un dedo necesitamos unir las falanges con **articulaciones**, que definen el origen de su unión y el tipo de movimiento que ocurre entre ellos. Entonces, deberíamos definir una articulación entre la falange proximal e intermedia, y luego otra entre la intermedia y distal.

Además, cada articulación puede tener un **actuador** para controlarla. Esto se ve más adelante.

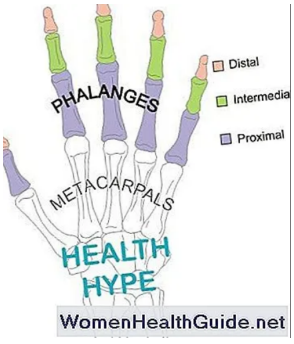


Figura 10: Divisiones de la mano

5.2.2. Carpeta de assets

Volviendo al directorio del FR3, la carpeta **assets** contienen varios archivos del tipo *.obj* (para geometría de colisión) y *.stl* (para geometría de visualización) que definen y muestran los cuerpos del robot.

Sobre colisión vs visualización: No necesariamente describen exactamente la misma geometría. Muchas veces la geometría de colisión de un objeto es mucho más simple que su geometría de visualización, ya que calcular colisiones es muy caro computacionalmente para geometrías complejas. *Por eso mismo en los videojuegos el modelo de un personaje suele ser lo que vemos en pantalla, pero su hitbox es una geometría mucho más simple (normalmente un cubo o una esfera).*

5.2.3. scene.xml

Este archivo define una escena simple donde se levanta el robot en un entorno vacío. Se incluye el archivo **fr3.xml** y luego se configura el skybox y el piso para la visualización. Es parecido a nuetro *.xml* del ejemplo 1, sólo que sin el cilindro y el cubo.

5.2.4. fr3.xml

En este .xml, **se define el robot completo** con sus geometrías y articulaciones.

Esta es la parte más interesante del ejemplo, asi que vamos a revisarlo más a detalle:

5.2.5. fr3.xml - Valores por defecto

El archivo parte seteando la configuración de compilador y el integrador, **como ya vimos en el ejemplo 1.**

```
<mujoco model="fr3">
  <!-- Parte 1 : Lo mismo que ex1 -->
  <compiler angle="radian" meshdir="assets"/>
  <option integrator="implicitfast"/>
```

Luego, se setean los **valores por defecto**. Estos son clases que tienen atributos que se repiten varias veces, asi se pueden "heredar.ª varios objetos en el mismo archivo.

Viendo el segmento de código de abajo vemos que, por ejemplo, la clase `"visual"` cambia el *contype* y *conaffinity* de forma que las geometrías visuales (que estarán bajo esta clase) no sean tomadas en cuenta para las colisiones.

```
  <default>
    <default class="fr3">
      <!-- Overwritten in joints 5, 6, and 7. -->
      <joint damping="0.21" armature="0.195" frictionloss="1.137"/>
      <position inheritrange="1"/>
      <default class="visual">
        <geom type="mesh" group="2" contype="0" conaffinity="0"/>
      </default>
      <default class="collision">
        <geom type="mesh" group="3" mass="0" density="0"/>
      </default>
```

```
    <site size="0.001" rgba="0.5 0.5 0.5 0.3" group="4"/>
  </default>
</default>
```

5.2.6. fr3.xml - Assets

En el siguiente segmento, se definen primero los `"materiales"` del robot. En otras palabras, los distintos colores que tiene su visualización.

Estos se definen como un nombre (para llamar el material) y luego su valor rgba. Por ejemplo:

```
<material name="black" rgba=".2 .2 .2 1"/>
```

Luego, se definen las geometrías a usar. Para esto, simplemente se definen los archivos a utilizar como *meshes*. En el caso de las geometrías de colisión, se les pone un nombre explícito.

```
<mesh file="link7_3.obj"/>

<mesh name="link0_coll" file="link0.stl"/>
```

5.2.7. fr3.xml - Definir el robot y articulaciones (Parte 1)

Este segmento es el más importante del archivo, pues se llaman las geometrías y se definen las articulaciones del robot. Realmente todo lo anterior es llamado en esta parte.

El brazo FR3 de Franka Emika es un brazo de varias partes y articulaciones, con varios grados de libertad. En esta parte del código, se establece la **cadena cinemática** del robot.

Cadena cinemática? – Complementemos un poco más la explicación de cómo se compone un robot.

5.2.8. Cómo se componen los robots en Mujoco (Explicación Nivel 2)

Para los que han dado el curso de Dinámica de Sistemas Mecánicos, sabrán que existen los marcos de referencia. Usando este concepto, Mujoco establece relaciones entre los distintos cuerpos del robot para calcular sus posiciones. Visto de cierta forma, cada robot es un **árbol de relaciones padre-hijo**, donde un **hijo se mueve en relación al marco de referencia del padre**, de forma acorde a la **articulación que los une**.

En otras palabras, cada cuerpo hijo define su **marco de referencia relativo al de su padre** mediante una posición y una orientación fijas. Sobre ese marco, la articulación (joint) que los une determina cómo puede moverse el hijo respecto del padre (por ejemplo, un giro alrededor de un eje). Si un cuerpo no tiene articulación, queda rígidamente fijo a su padre.

En el ejemplo de la mano, la falange proximal es padre de la intermedia, la falange intermedia es padre de la distal. Las articulaciones entre ellas definen cómo se puede mover acorde a sus marcos de referencia. De esta forma la posición de cualquier parte del robot en el mundo se obtiene componiendo en cadena todas las transformaciones desde la raíz hasta ese cuerpo (**cinemática directa**), y basta mover un eslabón para que todos sus descendientes lo acompañen automáticamente.

A nivel práctico, esto significa que vamos construyendo el robot desde el **marco de referencia del mundo** hacia el efector del robot (el efector de un brazo, por ejemplo, sería su mano). De adentro hacia afuera, de abajo hacia arriba.

Nota aparte: Este modelo de cinemática directa también facilita el control de robots por **cinemática inversa**, cosa que se usa mucho en industria y que vale la pena investigar.

5.2.9. fr3.xml - Definir el robot y articulaciones (Parte 2)

Ahora volvamos al archivo.

Se parte desde el marco de referencia del mundo o *worldbody* y se define el cuerpo *base*, al cual se le pasa `childclass="fr3"` para que todos sus cuerpos hijos hereden los atributos de la clase `"fr3"` definida antes. Con esto, se va construyendo iterativamente el robot como una ***cadena*** con *eslabones* o ***links***.

Para cada paso, primero se define el ***body***. Luego, se declara su información **inercial**, la **articulación** que define su movimiento o grados de libertad, sus **geometrías de colisión y visualización** y finalmente su **body hijo**. Asi se va declarando, iterativamente, cada eslabón de la cadena.

Para cada **body hijo**, se declara una **pose inicial**. Esta pose inicial se declara en el marco de referencia del eslabón padre. En otras palabras, se define la posición del origen del marco de referencia de cada eslabón en base al marco de referencia de su padre.

Por ejemplo, si el marco de referencia de mi padre es el origen y yo estoy declarado en una pose inicial de `(1,0,0)`, entonces mi pose inicial global será `(1,0,0)`. Luego, si tengo un hijo cuya pose inicial delcarada es `(1,0,0)`, como es mi hijo su pose inicial *global* será `(2,0,0)`, ya que hereda mi marco de referencia que ya está desplazado respecto al origen.

Luego, como ya se ha dicho antes, se declara la **articulación**. Mujoco tiene distintos tipos de ***joints*** que definen distintos tipos de movimiento. Por defecto, las *joints* son del tipo *hinge* (rotacional o bisagra), por lo que no se declara un *type* en este archivo. Además, se debe indicar el eje en que esta rotación ocurre, su rango de movimiento y su rango de fuerza para el actuador (que se declara más adelante en otra sección). Para ver todos los tipos de *joints*, recomiendo ver la documentación oficial de Mujoco.

Esto se puede ver en el siguiente segmento de código:

```
<body name="fr3_link1" pos="0 0 0.333">
  <inertial pos="4.128e-07 -0.0181251 -0.0386036" quat="0.998098 -0.0605364
  0.00380499 0.0110109" mass="2.92747"
    diaginertia="0.0239286 0.0227246 0.00610634"/>
  <joint name="fr3_joint1" axis="0 0 1" range="-2.7437 2.7437"
  actuatorfrcrange="-87 87"/>
  <geom name="fr3_link1_collision" class="collision" mesh="link1_coll"/>
  <geom material="white" mesh="link1" class="visual"/>
  <body name="fr3_link2" quat="1 -1 0 0">
    <inertial pos="0.00318289 -0.0743222 0.00881461" quat="0.502599 0.584437
    -0.465998 0.434366" mass="2.93554"
      diaginertia="0.0629567 0.0411924 0.0246371"/>
      .
      .
      .
```

En el caso del eslabón base, no se espera que este se mueva por lo que no se le asigna información inercial ni articulaciones. Esto hace que la base permanezca fija en su marco de referencia.

Para el último eslabón, en vez de llamar otro *body*, se declara un ***site*** llamdo ***attachment site***. La idea de esto es que se puedan adjuntar más robots a este sitio como, por ejemplo, una mano robótica *(spoiler: esto lo haremos en el próximo ejemplo).*

5.2.10. fr3.xml - Actuadores

Hasta ahora el robot existe, pero no tiene actuadores. Sin actuadores, no se puede controlar el robot. Actuadores = motores, en nuestro caso.

Cada actuador se asocia a una **articulación** en base a su nombre.

En este caso, todos los actuadores son del tipo `"position"`, lo que significa que son servomotores. Para ver todos los tipos de actuadores, puedes ver la documentación oficial de Mujoco.

También se le asignan valores a sus **constantes de control**, siendo estas *kp* y *kv*, lo que implica que los motores de este robot usan controladores *PD* (*kp* para control proporcional y *kv* para el derivativo).

Gracias a *inheritrange=1* del *default*, estos actuadores heredan los límites de sus articulaciones, tanto de movimiento como de fuerza.

```
  <actuator>
    <position class="fr3" name="fr3_joint1" joint="fr3_joint1" kp="4500" kv="450"/>
    <position class="fr3" name="fr3_joint2" joint="fr3_joint2" kp="4500" kv="450"/>
    <position class="fr3" name="fr3_joint3" joint="fr3_joint3" kp="3500" kv="350"/>
    <position class="fr3" name="fr3_joint4" joint="fr3_joint4" kp="3500" kv="350"/>
    <position class="fr3" name="fr3_joint5" joint="fr3_joint5" kp="2000" kv="200"/>
    <position class="fr3" name="fr3_joint6" joint="fr3_joint6" kp="2000" kv="200"/>
    <position class="fr3" name="fr3_joint7" joint="fr3_joint7" kp="2000" kv="200"/>
  </actuator>
```

5.2.11. fr3.xml - Exclusión de contacto

En la última parte del archivo, se excluye el contacto entre el primer y segundo eslabon (*link0* y *link1)* ya que están físicamente muy cerca y, sin esta exclusión, Mujoco detectaría que están en colisión constante (e irremediable). Los otros pares adyacentes no chocan por geometría, por lo que no es necesario excluirlos.

```
  <contact>
    <exclude body1="fr3_link0" body2="fr3_link1"/>
  </contact>
</mujoco>
```

5.3. Parte 3: Incluir el brazo en nuestra escena

Primero, necesitaremos duplicar nuestros archivos `e1_pt3.xml` y `e1_pt3.py` y cambiarles el nombre a `e2.xml` y `e2.py`.

5.3.1. e2.xml

Para incluir el robot en la escena, debemos hacer 2 cosas: importar el modelo y arreglar el path a los assets.

Para esto, modificamos lo siguiente al inicio del archivo (el resto queda exactamente igual a como estaba en el ejemplo anterior:

```
<mujoco model="Robot Scene">

  <!-- Ajustes del compilador -->
  <compiler angle="radian" coordinate="local" autolimits="true"/>
  <option integrator="implicitfast"/>

  <!-- Incluir el brazo robótico Franka FR3.-->
  <include file="franka_fr3/fr3.xml"/>
  <!-- Redefinir la dirección de los meshes -->
  <compiler meshdir="franka_fr3/assets"/>

  <!--Textura del mundo (skybox y piso)-->
  .
  .
  .
```

5.3.2. e2.py

Aqui, sólo debemos cambiar el nombre de la escena *.xml* cargada a `"e2.xml"`.

Si ejecutamos este script, el output en ventana será algo como lo siguiente:

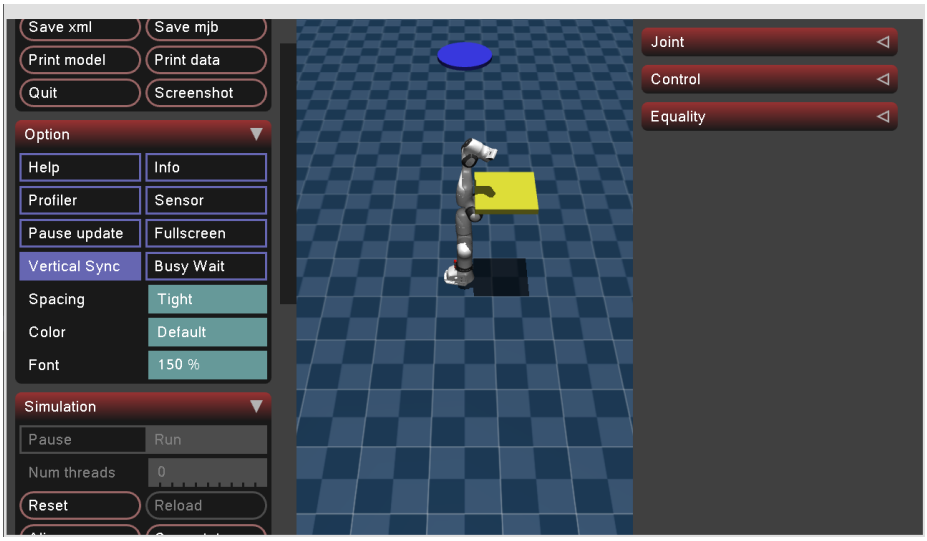


Figura 11: Output esperado en ventana del ejemplo 2

5.3.3. Controlando el robot (UI)

Viendo el panel de la derecha, podemos expandir la sección de `"control"`. Al hacerlo, se nos abren varios *sliders* con los nombres de nuestros actuadores. Esto significa que, desde este panel, podemos controlar directamente cada *servomotor* del brazo de forma directa desde la UI.

Juega con esto! Si todo está bien, deberías ver como el brazo robótico se mueve en el entorno y colisiona con los objetos a su alrededor.

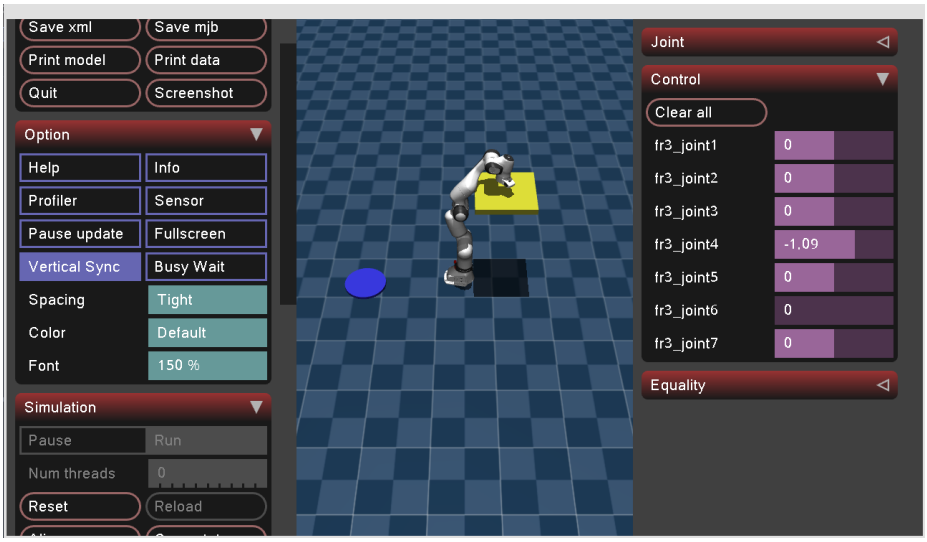


Figura 12: Resultado de controlar los actuadores del brazo

Por medio de **actuadores** es que se **controla el robot**. Se les da un *setpoint* y el controlador de cada *servomotor* se encarga de moverlo a esa posición. Esto se puede hacer **tanto por UI como por código** (normalmente, esa lógica va en el bloque comentado de `#LÓGICA#` en el script de Python.

5.3.4. Controlando el robot (código)

Para controlar un actuador desde el **código**, debemos modificar el bloque comentado de `#LÓGICA#` dentro de **e2.py**.

En Mujoco, al cargar el .XML del robot tenemos el **modelo (model)** y su **data (data)**. Por medio de esta última, podemos controlar el robot y sus actuadores, además de ver su posición.

Dentro del bloque comentado de `#LÓGICA#`, añade el siguiente comando:

```
data.actuator("fr3_joint2").ctrl = 1.0
```

Con esto, lo que acabamos de hacer es darle al **tercer actuador (fr3-joint2)** un **setpoint de 1.0**. Hay varias formas de hacer esto mismo *(las cuales se ejemplifican en el código del repositorio)*, pero esta es la más robusta ya que llama directamente al actuador por su nombre.

Ahora, si ejecutamos nuestro **e2.py**, veremos que el robot ajusta su pose para llegar al setpoint. Para ver la posición del joint asociado a este actuador, podemos usar el comando *data.joint("fr3-joint2").qpos* para obtener su posición.

Si intentamos modificar el setpoint desde la UI, no podremos ya que el comando se encuentra dentro del *while loop*. Si quisiéramos poder darle distintos setpoints por medio de la UI, habría que añadir la línea de código **antes del while loop.**

Con esto hemos terminado el ejemplo 2! Ahora ya sabemos cómo se construye un robot en Mujoco, añadirlo a nuestra escena y controlarlo por la UI.

6. Ejemplo 3: Ensamble de Robots

Ahora vamos a hacer un ensamble de robots! En otras palabra, juntar dos robots en la misma escena. Para eso, vamos a **darle una mano** a nuestro brazo robótico.

6.1. Descargar el modelo de la mano.

Volviendo a Mujoco Menagerie, ahora tenemos que descargar el directorio `shadow_hand` a nuestro repositorio.

Este directorio tiene exactamente la misma estructura de archivos que el del brazo robótico, como todos los robots del *Menagerie*. Útil!

Además incluye un archivo *keyframes* con posiciones predefinidias para que la mano realice ciertas acciones. Estas keyframes se pueden llamar desde el código (aunque no lo haremos en este ejemplo).

6.2. Escena

Para este ejemplo, volveremos a usar la escena del **ejemplo 1 parte 3** (`e1_pt3.xml`).

- Por que este paso atrás?

Porque ahora vamos a **ensamblar**, parte por parte, nuestra escena **en Python**.

6.3. e3.py

Para este código, vamos a cargar cada archivo .XML (`e1_pt3, fr3.xml, shadow_hand`) como un **MjSpec**.

- Qué es un MjSpec?

Básicamente, es la representación editable de un modelo en memoria. Luego, cuando se compila este MjSpec, se conviernte en un **MjModel**, que es el modelo inmutable del robot.

En general, el flujo de trabajo en Mujoco es:

$$XML \longrightarrow MjSpec \longrightarrow [compilar] \longrightarrow MjModel \longrightarrow MjData \text{ (estado del robot)}$$

Por eso, primero vamos a cargar cada .XML como un MjSpec dentro de nuestro código para luego combinarlos en un modelo cohesivo.

Nuestro script **e3.py**, luego de cargar las mismas librerías de los scripts ateriores, incluye lo siguiente:

```
# 1. Cargar las tres piezas como especificaciones independientes
PARENT = Path(__file__).resolve().parent
scene = mujoco.MjSpec.from_file(str(PARENT / "e1_pt3.xml"))               # ambiente
arm   = mujoco.MjSpec.from_file(str(PARENT / "franka_fr3" / "fr3.xml"))     # brazo
hand  = mujoco.MjSpec.from_file(str(PARENT / "shadow_hand" / "right_hand.xml"))  # mano

# 2. Atar la mano al site de montaje del brazo
# --> prefix="rh_" evita choques de nombres (ej: materiales "black"/"white")
arm.attach(hand, prefix="rh_", site=arm.site("attachment_site"))
```

```
# 3. Insertar el ensamble (brazo + mano) en el ambiente.
# Lo anclamos a un frame en el worldbody de la escena (en el origen).
mount = scene.worldbody.add_frame()
scene.attach(arm, prefix="", frame=mount)

# 4. Compilar la especificación combinada a un modelo utilizable
model = scene.compile()
data = mujoco.MjData(model)
```

Luego de esto, el loop de simulación es el mismo de los pasos anteriores.

Ahora, si ejecutamos el archivo, deberíamos ver en nuestra escena los objetos dinámicos y estáticos, además del brazo robótico en el origen y con una mano montada en su extremo efector.

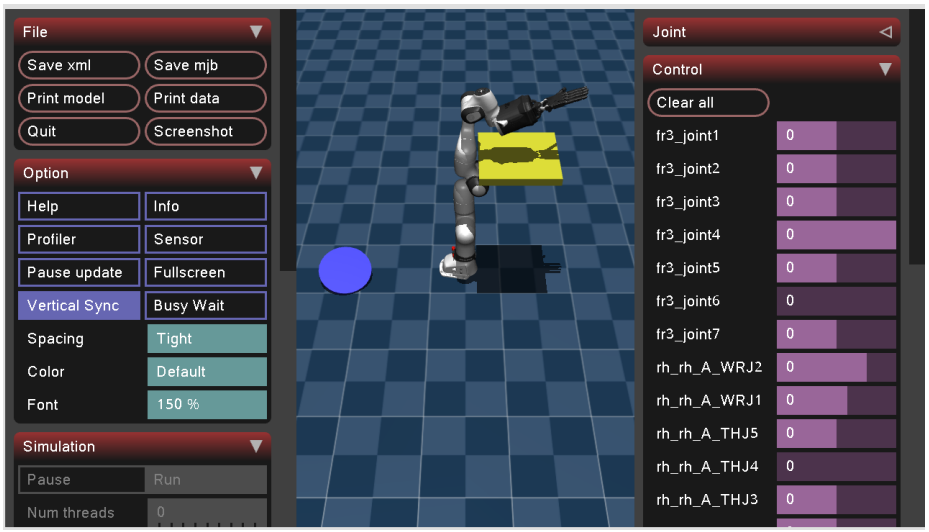


Figura 13: Simulación del ensamble

Desde la UI, también vemos que ahora podemos controlar tanto los actuadores del brazo como los de la mano robótica que le añdimos.

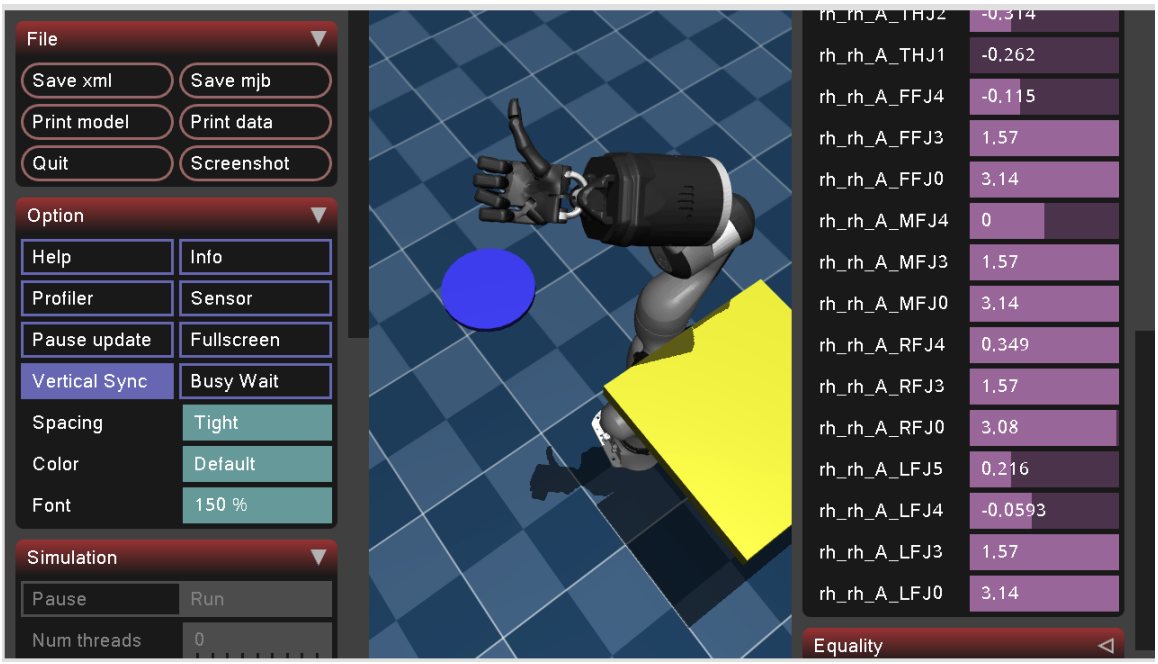


Figura 14: Demostración de control de ambos robots

Con esto, hemos terminado el ejemplo 3 y todo el manual! Ahora ya sabemos cómo es el workflow de compilar un modelo de robot en Mujoco y cómo montar nuestro propio ensamble de robots.

7. Conclusión

En este documento aprendieron las siguientes habilidades prácticas:

1. Desplegar una escena del simulador con objetos estáticos y dinámicos.
2. Incorporar un sistema articulado (robot) a una escena y controlarlo.
3. Generar un ensamble de sistemas robóticos en la simulación y controlarlo.

Hay muchas varias funcionalidades de Mujoco que este Manual no toca. Sin embargo, es buena puerta de entrada para seguir aprendiendo y usando este software para sus simulaciones, cursos e investigaciones.

Espero que hayan quedado claras las explicaciones y les sirvan en sus estudios y desarrollo de simulaciones. Realmente Mujoco no es un simulador tan complicado una vez lo entiendes, sólo hay que dedicarle tiempo (como todo en la vida...).

Espero que el manual les sirva!

8. Fuentes y Referencias

- DeepMind. (2022). MuJoCo Documentation. DeepMind. https://mujoco.readthedocs.io/
- DeepMind. (2023). MuJoCo Menagerie [Código fuente]. GitHub. https://github.com/google-deepmind/mujoco_menagerie
- Torres, M. (2024). SSMM [Código fuente]. GitHub. https://github.com/RAL-UC/SSMM/
- Cherubini, V. (2025). mujoco-manual [Código fuente]. GitHub. https://github.com/vittocher/mujoco-manual
- Bhounsule, Pranav A. (2022). MuJoCoPy Bootcamp. Recuperado 7 de octubre de 2025, de https://pab47.github.io/mujocopy.html